Nurarys Baibolat CS-2413

OSS Practice

1. Deploy an application inside a container with no resource limits and observe how it affects system memory and CPU usage over time.

```
[analyst@secOps ~]$ sudo docker run -d --name stress-testt   alpine sh -c "apk a
dd --no-cache stress-ng && stress-ng --cpu 4 --io 2 --vm 2 --vm-bytes 1G --timeo
ut 300s"
05a1592cece582c30b93538aa463cee4575bb8218d4626565718e8c7000a8391
```

```
File  Edit  View  Search  Terminal  Help
CONTAINER ID   NAME          CPU %       MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O     PIDS
6179e9f8018f   stress-test   23801.90%   76.73MiB / 3.366GiB   2.23%     6.26MB / 31.7kB   0B / 5.87MB   11
```

```
File  Edit  View  Search  Terminal  Help
top - 09:16:13 up 8 min,  1 user,  load average: 7.81, 3.18, 1.20
Tasks: 176 total,   9 running, 167 sleeping,   0 stopped,   0 zombie
%Cpu(s): 74.2 us,  3.6 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi, 22.2 si,  0.0 st
MiB Mem :   3446.6 total,   2462.6 free,    443.1 used,    540.8 buff/cache
MiB Swap:   2048.0 total,   2046.8 free,      1.2 used.   2753.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2325 root      20   0  546572  24744   1044 R  96.4   0.7   1:28.32 stress-ng-vm
   2315 root      20   0   22276   4128   2300 R  90.4   0.1   1:19.72 stress-ng-cpu
   2323 root      20   0  546572   6792   1044 R  65.2   0.2   1:19.05 stress-ng-vm
   2318 root      20   0   22276   4128   2300 R  62.3   0.1   1:29.45 stress-ng-cpu
   2316 root      20   0   22276   4128   2300 R  37.7   0.1   1:18.46 stress-ng-cpu
   2317 root      20   0   22276   4128   2300 R  33.8   0.1   1:27.26 stress-ng-cpu
```

2. Use system tools to identify which user and PID a containerized process runs under, and explain why this matters for security.

```
[analyst@secOps ~]$ sudo docker inspect 89869da8144d  | grep Pid
            "Pid": 2913,
            "PidMode": "",
            "PidsLimit": null,
[analyst@secOps ~]$ top -H -p 2913

top - 09:33:22 up 25 min,  1 user,  load average: 8.73, 6.52, 4.55
Threads:   1 total,   0 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s): 74.9 us,  2.2 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi, 22.9 si,  0.0 st
MiB Mem :   3446.6 total,   2354.7 free,    465.1 used,    626.7 buff/cache
MiB Swap:   2048.0 total,   2043.8 free,      4.2 used.   2728.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2913 root      20   0   22276  10152   9684 S   0.0   0.3   0:00.03 stress-ng
```

This is important because the container is a regular process on the host. Knowing his PID and user, you can understand what privileges he has. If it is running from root, then when hacking and exiting the container, an attacker can gain full control over the system. If it's from a regular user, the damage will be limited.

3. Compare the process visibility of a container versus the host system and describe what isolation mechanism is responsible.

Only its own processes are visible in the container, and all processes, including container processes, are visible on the host. This is provided by the isolation mechanism **PID namespace**, which creates a separate process identifier space for the container.

4. Apply memory and CPU limits to a running container and analyze how these restrictions change its behavior under load.

```
[analyst@secOps ~]$ sudo docker run -d --name stress-limited \
  --memory="256m" \
  --cpus="0.5" \
  alpine sh -c "apk add --no-cache stress-ng && stress-ng --cpu 2 --vm 1 --vm-bytes 1G --timeout 300s"
d68c042c21b94361759c6d66b485e056c9e4d24efe67e241e22732bf5174363d
[analyst@secOps ~]$ docker stats
top
free -m
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http
://%2Fvar%2Frun%2Fdocker.sock/v1.50/containers/json": dial unix /var/run/docker.sock: connect: permission denie
d

top - 09:41:26 up 33 min,  1 user,  load average: 0.01, 1.98, 3.29
Tasks: 172 total,   5 running, 167 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.2 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3446.6 total,   2094.2 free,    723.1 used,    629.3 buff/cache
MiB Swap:   2048.0 total,   1871.1 free,    176.9 used.   2470.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    968 root      20   0  413576 117884  70672 S   2.7   3.3   0:28.08 Xorg
   2349 analyst   20   0  832016  51104  39720 S   1.0   1.4   0:05.20 mate-terminal
```

5. Explain how memory exhaustion by a single process could impact an endpoint if no isolation mechanisms are in place.

```
[analyst@secOps ~]$ stress-ng --vm 1 --vm-bytes 90% --timeout 60s
stress-ng: info:  [3437] setting to a 60 second run per stressor
stress-ng: info:  [3437] dispatching hogs: 1 vm
stress-ng: info:  [3437] successful run completed in 60.34s (1 min, 0.34 secs)
top - 09:46:56 up 39 min,  1 user,  load average: 0.68, 1.00, 2.44
Tasks: 172 total,   2 running, 170 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3446.6 total,    242.5 free,   2443.9 used,    760.1 buff/cache
MiB Swap:   2048.0 total,   2044.3 free,      3.7 used.    749.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3449 analyst   20   0   10472   3956   3268 R 100.0   0.1   0:00.01 top
      1 root      20   0  167116  12372   8132 S   0.0   0.4   0:01.63 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
```

6. Identify at least three risks of running a container with elevated privileges on a production endpoint.

Running a container with elevated privileges is dangerous because it weakens isolation. The container can access host devices, change kernel or firewall settings, and, if compromised, an attacker can escape the container more easily and gain root access on the host. This greatly increases the risk of full system compromise.

7. Harden a container by removing unnecessary permissions and explain how this reduces the attack surface.

```
[analyst@secOps ~]$ sudo docker run -d \
  --read-only \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --pids-limit 100 \
  --memory=256m \
  nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
c7b840964e41bd0f3ecaaf94126c22169d343677596399c31c3067b80f920816
```

A container is hardened by removing unnecessary permissions, such as dropping capabilities, running as a non-root user, using a read-only file system, and enabling no-new-privileges. This reduces the attack surface because an attacker who compromises the container has fewer actions they can perform and limited ability to affect the host system.

8. Analyze how read-only file systems help prevent persistence after exploitation.

A read-only file system helps prevent persistence because an attacker who exploits a container cannot modify or create files inside it. They cannot install malware, change system binaries, alter configurations, or place backdoors for future access. As a result, any malicious activity is temporary and disappears after the container restarts, which prevents long-term compromise.

9. Describe which exploit techniques are mitigated by process isolation and which ones still remain possible.

Process isolation limits many attacks by separating processes into different namespaces, so they cannot see or interfere with processes on the host or in other containers. This helps prevent techniques like process injection, ptrace abuse, sending malicious signals, or accessing other processes' memory. However, it does not stop kernel exploits, container escape vulnerabilities, application-level attacks, network threats, or denial-of-service through resource exhaustion. So isolation reduces cross-process attacks but does not fully remove security risks.

10. As a SOC analyst, decide what manual investigation steps are required when an isolated process shows suspicious behavior.

```
[analyst@secOps ~]$ ps auxf
docker top <container>
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           2  0.0  0.0      0     0 ?        S    09:07   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   09:07   0:00  \_ [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   09:07   0:00  \_ [rcu_par_gp]
root           5  0.0  0.0      0     0 ?        I<   09:07   0:00  \_ [slub_flushwq]
root           6  0.0  0.0      0     0 ?        I<   09:07   0:00  \_ [netns]
root           8  0.0  0.0      0     0 ?        I<   09:07   0:00  \_ [kworker/0:0H-events_highpri]
root          10  0.0  0.0      0     0 ?        I<   09:07   0:00  \_ [mm_percpu_wq]
root          11  0.0  0.0      0     0 ?        S    09:07   0:00  \_ [rcu_tasks_rude_]
root          12  0.0  0.0      0     0 ?        S    09:07   0:00  \_ [rcu_tasks_trace]
[analyst@secOps ~]$ netstat -tunlp
ss -plant
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:35645         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::23                   :::*                    LISTEN      -
udp        0      0 127.0.0.53:53           0.0.0.0:*                           -
```

```
[analyst@secOps ~]$ journalctl
Hint: You are currently not seeing messages from other users and the system.
      Users in groups 'adm', 'systemd-journal' can see all messages.
      Pass -q to turn off this notice.
Feb 10 21:25:19 labvm dbus-daemon[72440]: [session uid=1002 pid=72438] AppArmor D-Bus mediation is enabled
Feb 10 21:25:19 labvm dbus-daemon[72440]: [session uid=1002 pid=72438] Activating service name='ca.desrt.dconf
Feb 10 21:25:19 labvm dbus-daemon[72440]: [session uid=1002 pid=72438] Successfully activated service 'ca.desr
Feb 10 21:27:44 labvm dbus-daemon[72440]: [session uid=1002 pid=72438] Reloaded configuration
Feb 10 21:27:44 labvm dbus-daemon[72440]: [session uid=1002 pid=72438] Reloaded configuration
Feb 10 21:27:44 labvm dbus-daemon[72440]: [session uid=1002 pid=72438] Reloaded configuration
```

```
[analyst@secOps ~]$ sudo docker diff 89869da8144d
C /lib
C /lib/apk
C /lib/apk/db
C /lib/apk/db/triggers
C /lib/apk/db/installed
A /lib/apk/db/lock
C /lib/apk/db/scripts.tar.gz
C /etc
C /etc/apk
C /etc/apk/world
```

```
[analyst@secOps ~]$ sudo docker inspect 89869da8144d
[
    {
        "Id": "89869da8144d005ab1eeca766f1f33c28774ea02913bde3a87ca6c72a974f727",
        "Created": "2026-01-27T09:29:54.577250769Z",
        "Path": "sh",
        "Args": [
            "-c",
            "apk add --no-cache stress-ng && stress-ng --cpu 4 --io 2 --vm 2 --vm-bytes 1G --timeout 300s"
        ],
        "State": {
            "Status": "exited",
            "Running": false,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
```

```
[analyst@secOps ~]$ sudo docker logs 89869da8144d
(1/5) Installing judy (1.0.5-r1)
(2/5) Installing libmd (1.1.0-r0)
(3/5) Installing libbsd (0.12.2-r0)
(4/5) Installing liblksctp (1.0.19-r5)
(5/5) Installing stress-ng (0.19.03-r0)
Executing busybox-1.37.0-r30.trigger
OK: 13.6 MiB in 21 packages
stress-ng: info:  [1] setting to a 5 mins run per stressor
stress-ng: info:  [1] dispatching hogs: 4 cpu, 2 io, 2 vm
stress-ng: info:  [14] io: this is a legacy I/O sync stressor, consider using iomix instead
stress-ng: info:  [16] vm: using 512M per stressor instance (total 1G of 2.38G available memory)
```

A SOC analyst should manually check running processes and their parent-child relationships, review network connections for suspicious traffic, examine file system changes, and analyze container and host logs. They should also verify the container image source, check the process user and privilege level, and collect hashes of suspicious files for further analysis. These steps help determine if the activity is malicious and assess the impact.